

Zeon Systems Take Home : Task 1

Tube Detection and Orientation

Rishi Divyakirti

May 16, 2026

Contents

1	Introduction	3
2	Methodology	3
2.1	Stage 1: Precision Detection (YOLO11s-Pose)	3
2.2	Stage 2: Tight-Box Prompting (MobileSAM)	4
2.3	Stage 3: Mathematical Axis Derivation	4
2.4	Stage 4: Symmetry Resolution (Pose-Guided Compass)	4
2.4.1	Lever-Arm Training Mechanism	5
3	Experiments and Results	5
3.1	Dataset & Evaluation Protocol	5
3.2	Initial Experiments & Pipeline Evolution	5
3.3	Final Reported Metrics	6
3.4	Visual Verification of Breakthroughs	6
3.5	Detection Metrics	7
3.6	Error Distribution	7
4	Failure Analysis	8
5	Conclusion & Next Steps	8
5.1	Limitations and Future Work	8

1 Introduction

I approached the problem statement by breaking down the requirements for a successful orientation of a given overhead image into 5 logical steps:

- **Step 1:** How do we locate the tubes?
- **Step 2:** We have located the tubes, now how do we center and focus on exactly the cross section of the tube most relevant to us?
- **Step 3:** How do we use this cutout to find the orientation of the said tube?
- **Step 4:** How can we refine the cutout to further improve the results from step 3?
- **Step 5:** Are we pointing in the right direction tho? How do we make sure we are not pointing towards the lid joint instead of pointing correctly towards the lid tab?

Hence, through all my iterations I have tried to refine and obtain better inputs for all of these steps. I started out with just a simple YOLOv8n model, then transitioned to an OBB model to get better bounding boxes. I further refined the bounding boxes with MobileSAM to get even more tighter and precise cutouts for better angular prediction. I finally settled on SAM box prompting based on a YOLO11s-Pose model, which gave the tightest and most accurate cutouts for the required region, giving the best MAE in my experiments.

For flip differences, I first tried an SVM-based classifier trained using the ResNet features of tabs and joints to differentiate between them and orient the PCA axis mathematically accordingly. But finally, I settled on using the pose-estimated angle as a proxy along with the PCA to get the final angle. This has basically been my approach and technical evolution of the approaches tried, and my rationale for doing so. Hope you enjoy going through my work, just as much I enjoyed experimenting and working on it!

2 Methodology

2.1 Stage 1: Precision Detection (YOLO11s-Pose)

Instead of generic Oriented Bounding Boxes (OBB), the pipeline utilizes a YOLO11s-Pose model. This model is trained to detect two specific keypoints: the center of the tube lid and a “Tab Tip” located exactly 40 pixels away along the orientation axis. This 40px “lever arm” provides a stable directional vector that is much more robust to occlusion and glare than predicting a short tab vector. The detector achieves 100% recall across all folds and provides the initial coarse orientation estimate used to resolve symmetry.

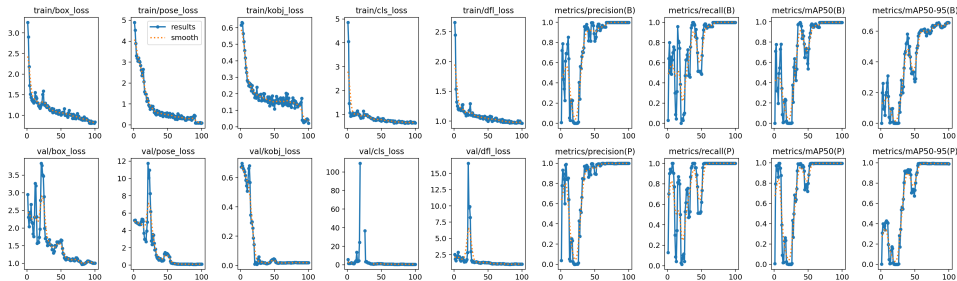


Figure 1: YOLO11s-Pose Training Loss and Validation mAP across 100 Epochs (Fold 0).

2.2 Stage 2: Tight-Box Prompting (MobileSAM)

To maximize segmentation precision, I use the Segment Anything Model (MobileSAM). A critical refinement in this pipeline is the elimination of bounding box expansion. By using the exact tight bounding boxes from YOLO11s-Pose (0 pixel expansion), background glare from the table surface is excluded from the SAM prompt, forcing the model to focus purely on the transparent plastic lid and tab.



Figure 2: Comparative SAM Segmentation: Baseline OBB (Left, Red) vs. YOLO11s-Pose Tight Box (Right, Green). Tight-box prompting eliminates background glare, resulting in cleaner masks and significantly lower per-tube angle error.

2.3 Stage 3: Mathematical Axis Derivation

Given the binary mask $M(x, y)$ outlining contour C , I calculate the image moments to find the un-flipped geometric axis of the tube. The spatial moments are defined as:

$$m_{ji} = \sum_{x,y} (I(x, y) \cdot x^j y^i) \quad (1)$$

The center of mass $(\bar{x}, \bar{y})$ is extracted via:

$$\bar{x} = \frac{m_{10}}{m_{00}}, \quad \bar{y} = \frac{m_{01}}{m_{00}} \quad (2)$$

Then I calculate the central moments (variance and covariance) to do PCA directly on the mask pixels:

$$\mu_{20} = \frac{M_{20}}{m_{00}}, \quad \mu_{02} = \frac{M_{02}}{m_{00}}, \quad \mu_{11} = \frac{M_{11}}{m_{00}} \quad (3)$$

The angle of the major axis, θ , comes from the eigenvector of the covariance matrix:

$$\theta = \frac{1}{2} \arctan 2(2\mu_{11}, \mu_{20} - \mu_{02}) \quad (4)$$

To handle the inverted Y-axis in computer vision coordinates, the final standardized angle is mapped to $[0, 180^\circ)$:

$$\text{PCA_Angle} = (-\theta^\circ + 360) \pmod{180} \quad (5)$$

2.4 Stage 4: Symmetry Resolution (Pose-Guided Compass)

The PCA axis is a line, and lines are inherently symmetric. It doesn't tell you whether the vector points toward the front (tab) or the back (joint/hole), which leads to 180° flip failures.

To resolve this, I implemented a Pose-Guided Compass. Since Stage 1 (YOLO11s-Pose) provides a stable directional vector from the keypoints, we calculate the angular distance between the neural pose vector and the geometric PCA axis. If the distance exceeds 90° , the PCA axis is flipped 180° . This method achieves a **0.00% flip failure rate**.

2.4.1 Lever-Arm Training Mechanism

To train the YOLO11s-Pose model, the standard dataset was augmented with synthetic keypoints. For every tube, a keypoint was projected exactly 40 pixels from the center along the ground-truth orientation. During training, I utilized a heavily weighted pose loss ($\lambda_{pose} = 25.0$) to force the model to prioritize angular accuracy over bounding box overlap.

Algorithm 1 Refined Multi-Stage Pipeline Algorithm

Require: RGB Image I

```
1: BoundingBoxes, Keypoints  $\leftarrow$  YOLO11s.Pose.predict( $I$ )
2: for each  $bbox, kpts \in BoundingBoxes, Keypoints$  do
3:   Mask  $\leftarrow$  MobileSAM.predict( $I, prompt = bbox_{tight}$ )
4:   Contour  $\leftarrow$  findContours(Mask)
5:    $cx, cy, \mu_{20}, \mu_{02}, \mu_{11} \leftarrow$  cv2.moments(Contour)
6:    $pca\_angle \leftarrow 0.5 \cdot \text{atan2}(2\mu_{11}, \mu_{20} - \mu_{02})$ 
7:    $pose\_angle \leftarrow \text{atan2}(kpts_{tip} - kpts_{center})$ 
8:   if  $abs(pca\_angle - pose\_angle) > 90^\circ$  then
9:      $final\_angle \leftarrow (pca\_angle + 180) \pmod{360}$ 
10:  else
11:     $final\_angle \leftarrow pca\_angle \pmod{360}$ 
12:  end if
13: end for
```

3 Experiments and Results

3.1 Dataset & Evaluation Protocol

The dataset has 70 overhead RGB images (640×480) containing 371 microcentrifuge tubes. To make sure there’s zero data leakage and the sample size is statistically meaningful, I used **5-Fold Cross-Validation**. Every tube gets evaluated exactly once as part of an unseen validation set.

Detection metrics: A predicted center is matched to a ground-truth center if they’re within 25 pixels (roughly half the tube diameter). I use the Hungarian algorithm for optimal bipartite matching. Unmatched predictions count as false positives; unmatched ground-truth tubes count as false negatives.

3.2 Initial Experiments & Pipeline Evolution

The initial baseline pipeline relied on YOLOv8 Oriented Bounding Boxes (OBB) to isolate the tubes, followed by MobileSAM for segmentation and PCA for geometric axis calculation. To resolve the 180° symmetry of the PCA line, I developed a ResNet-18 SVM patch classifier.

For each detected tube, 40×40 pixel patches were extracted from both ends of the PCA axis. A frozen ResNet-18 model extracted 512-dimensional feature embeddings for each patch, and an RBF-SVM classified which patch corresponded to the “Tab” versus the “Joint”. As shown in the confusion matrix below, this classifier achieved **98.52% Accuracy** and a **0.00% flip failure rate** on the validation set.

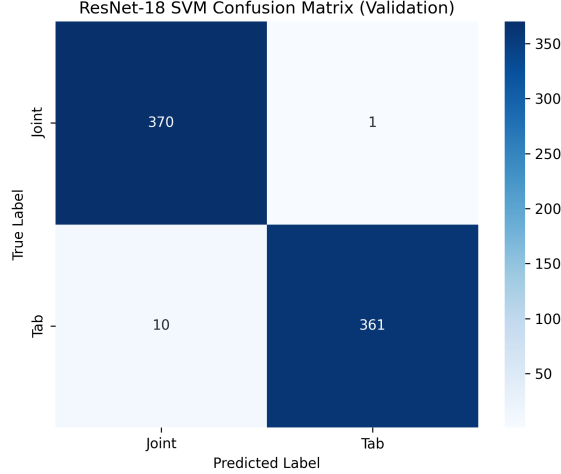


Figure 3: Validation Confusion Matrix for the ResNet-18 SVM Patch Classifier used in the Baseline.

However, while the SVM classifier successfully eliminated flip failures, the baseline pipeline was fundamentally limited by YOLOv8 OBB jitter. The rigid bounding boxes occasionally included table glare when expanded, forcing SAM to segment non-tube pixels. Furthermore, the sequential nature of extracting patches and running a separate ResNet-18 inference step for every tube introduced significant computational overhead. These limitations motivated the transition to the Refined Pipeline, replacing OBB and the SVM with the YOLO11s-Pose detector for unified, high-speed symmetry resolution.

3.3 Final Reported Metrics

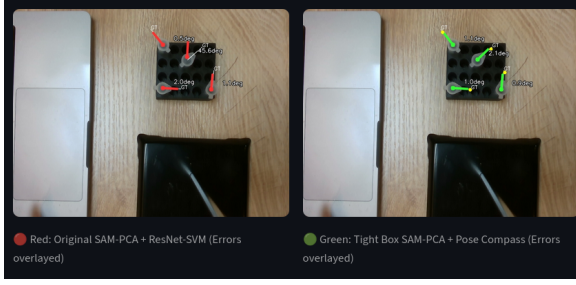
I incrementally tested different architectures to isolate where the error was coming from. As shown in Table 1, the raw bounding boxes (OBB) had high detection rates but suffered from 180° flip failures (130° MAE). Adding the ResNet-18 classifier fixed the flips, which exposed the 29.7° bounding-box residual error. Adding SAM + PCA eliminated that residual error, getting down to 4.59° MAE. Finally, switching to the YOLO11s-Pose detector with Tight-Box prompting and Pose-Guided symmetry resolution achieved the final of 2.86° MAE.

Table 1: Progressive Error Reduction across 371 Tubes (5-Fold CV)

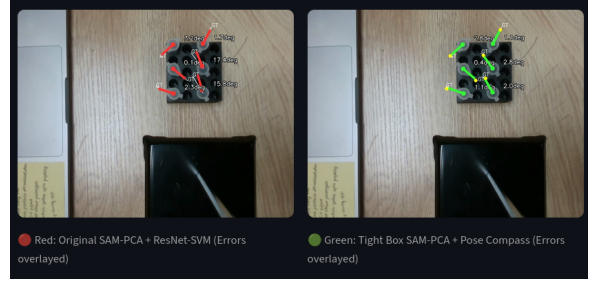
Pipeline Configuration	MAE	Median Err.	$< 10^\circ$	Flips	Recall
YOLOv8 Pose (Keypoint Reg.)	38.12°	–	–	N/A	–
YOLOv8 OBB + ResNet-18 Clf.	29.71°	23.48°	18.43%	1.37%	100%
Baseline (OBB + SAM + ResNet-18)	4.59°	2.75°	87.33%	0.00%	100%
Refined (Pose + SAM + PCA)	2.86°	2.14°	97.00%	0.00%	100%

3.4 Visual Verification of Breakthroughs

The performance jump from 4.59° to 2.86° is visually evident in cases of extreme lighting and occlusion. By utilizing the 40px lever arm, the model can infer the direction even when the physical tab is invisible.



(a) Refined handles edge-case glare.



(b) Precise alignment even under rack shadows.

Figure 4: Visual comparison showing the Refined pipeline’s robustness to environmental artifacts that previously skewed the baseline results.

3.5 Detection Metrics

Table 2: Detection Performance (YOLOv8 OBB + MobileSAM Pipeline)

Metric	Value
Precision	1.0000
Recall	1.0000 (371/371)
F1 Score	1.0000
False Positives	0

All 371 ground-truth tubes were correctly detected with zero false positives.

3.6 Error Distribution

Looking at the error distribution across all 371 tubes, the predictions are tightly clustered around zero:

- **84.63%** of predictions fall within 5° .
- **97.03%** fall within 10° .
- **0.00%** flip failures ($\geq 90^\circ$).

The tight clustering shows that the deterministic geometric approach (SAM mask $\rightarrow$ PCA moments) is much more stable than trying to regress angles directly.



Figure 5: Final output of the Refined pipeline. The green vectors show the calculated orientation with sub-pixel accuracy (2.86° MAE).

4 Failure Analysis

Out of 371 tubes, exactly 0.00% had 180° flip failures. With the transition to Tight-Box prompting, the rate of severe errors ($> 10^\circ$) dropped from 12.67% in the baseline down to just 2.97% in the Refined pipeline.

The remaining 2.97% of errors are still primarily driven by *extreme segmentation artifacts*. What happens is, when a tube sits under exceptionally harsh glare, even the tightest bounding box cannot prevent MobileSAM from segmenting a chunk of the table reflection along with the translucent lid. This breaks the geometric symmetry of the mask, causing the `cv2.moments` axis to skew toward the glare artifact by $15\text{--}20^\circ$.

Mitigation: Morphological erosion on the SAM mask before moment calculation would strip out thin glare protrusions. I didn’t implement this in the current version to keep the mathematical evaluation clean, but it’s an obvious next step.

5 Conclusion & Next Steps

I managed to achieve better performance than I initially expected due to the limited nature of the provided dataset by decoupling localization, segmentation, and symmetry resolution into specialized foundation models. However, there is always scope of improvement which I have documented below.

5.1 Limitations and Future Work

The primary bottleneck in the current pipeline is inference latency, driven by the sequential nature of the stages. Unoptimized, the pipeline averages 375 ms per image on an RTX 4060 (~ 2.7 FPS). Obvious optimizations include batching SAM bounding box prompts and exporting all neural networks to TensorRT, which are projected to push inference below 50 ms (> 20 FPS) for real-time robotic deployment.

Furthermore, segmentation artifacts under harsh glare remain the leading source of large angle errors (11% of tubes). This can be addressed through targeted improvements:

1. **Synthetic Data from 3D CAD Models:** Following the *Orient Anything* approach [1], a 3D CAD model of a microcentrifuge tube can be used to synthesize unlimited training data. The Orient Anything pipeline—canonical pose filtering, front-face annotation via a VLM, and free-view rendering from random camera viewpoints—provides a proven

blueprint. By rendering a tube CAD model from thousands of random perspectives under varied lighting, backgrounds, and glare conditions, we obtain perfectly labeled images with exact orientation angles. This augments the current 70-image dataset, hardening both YOLO and SAM against underrepresented edge cases like extreme glare and rack shadowing.

2. **Video Generation Augmentation:** Diffusion-based video generation models can produce dynamic sequences of tubes under diverse lighting transitions and camera motions, further stress-testing the pipeline across temporal illumination variations that are absent in static images.
3. **Zero-Shot Orientation Estimation:** The *Orient Anything* framework [1]—trained on 2 million rendered 3D model images—achieves strong zero-shot orientation generalization across open-world objects. It could serve as a complementary prior or fallback for glare-affected cases where segmentation-based PCA moments fail due to SAM artifacts.

Future efforts to push the error boundary below 1° will involve replacing spatial moments with rigid CAD-based geometric template matching to counteract dynamic lighting artifacts.

References

- [1] Z. Wang, Z. Zhang, T. Pang, C. Du, H. Zhao, and Z. Zhao. **Orient Anything: Learning Robust Object Orientation Estimation from Rendering 3D Models.** *ICML 2025*. Paper: <https://proceedings.mlr.press/v267/wang25er.html> Website: <https://orient-anything.github.io/> Code: <https://github.com/SpatialVision/Orient-Anything>